Theory of Computation

Lecture 3: Deterministic Finite Automata

Sheikh Azizul Hakim
Lecturer, Department of CSE, BUET

Last compiled: June 22, 2026 3:01am Z

How do we describe finite memory precisely?

3A. The Concept of State & Mathematical Models

Where We Are in the Story

Lecture	Question	Main idea
1	Why study computation?	Limits of machines
2	What are problems?	Languages and finite memory intuition
3	How do we specify memory?	Deterministic finite automata
4	What if machines can guess?	Nondeterminism (Next)

Today we turn intuition into a precise mathematical object.

What is a “State”?

Before formalizing automata, we must understand a **state**.

What is a “State”?

Before formalizing automata, we must understand a **state**.

A system interacts with a sequence of events over time. It cannot (and should not) remember every single event that ever happened.

What is a "State"?

Before formalizing automata, we must understand a **state**.

A system interacts with a sequence of events over time. It cannot (and should not) remember every single event that ever happened.

Core Concept
A **state** is a compressed summary of the past. It captures *exactly* the information needed to determine future behavior, ignoring irrelevant history.

Everyday Example: A Traffic Light

Consider a simple automated traffic light at an intersection.

Everyday Example: A Traffic Light

Consider a simple automated traffic light at an intersection.

- Does it remember how many cars passed yesterday? **No.**
- Does it remember the exact time of the last green light? **No.**

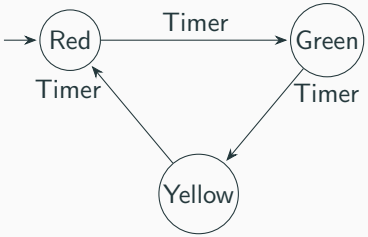
Everyday Example: A Traffic Light

Consider a simple automated traffic light at an intersection.

- Does it remember how many cars passed yesterday? **No.**
- Does it remember the exact time of the last green light? **No.**

It only needs to remember its current phase (state) to know what to do next:

1. Red: Stop.
2. Green: Go.
3. Yellow: Prepare to stop.



Communicating a Machine

Suppose you designed a password validator or a tokenizer based on states.

How can you communicate the design to someone else?

Communicating a Machine

Suppose you designed a password validator or a tokenizer based on states.

How can you communicate the design to someone else?

- A drawing can be intuitive.

Communicating a Machine

Suppose you designed a password validator or a tokenizer based on states.

How can you communicate the design to someone else?

- A drawing can be intuitive.
- A mathematical specification is precise.

Communicating a Machine

Suppose you designed a password validator or a tokenizer based on states.

How can you communicate the design to someone else?

- A drawing can be intuitive.
- A mathematical specification is precise.
- A table can define one part of the mathematics.

In theory, we need descriptions that are clear, compact, and unambiguous.

Two Views of the Same Machine

Good for intuition and design

Good for proof and exact communication

A transition table is simply one convenient way to define the transition function.

A DFA Has Five Ingredients

To describe a deterministic finite automaton, we need:

A DFA Has Five Ingredients

To describe a deterministic finite automaton, we need:

1. a finite set of states,

A DFA Has Five Ingredients

To describe a deterministic finite automaton, we need:

1. a finite set of states,
2. an input alphabet,

A DFA Has Five Ingredients

To describe a deterministic finite automaton, we need:

1. a finite set of states,
2. an input alphabet,
3. a rule for moving between states,

A DFA Has Five Ingredients

To describe a deterministic finite automaton, we need:

1. a finite set of states,
2. an input alphabet,
3. a rule for moving between states,
4. a start state,

A DFA Has Five Ingredients

To describe a deterministic finite automaton, we need:

1. a finite set of states,
2. an input alphabet,
3. a rule for moving between states,
4. a start state,
5. a set of accepting states.

A DFA Has Five Ingredients

To describe a deterministic finite automaton, we need:

1. a finite set of states,
2. an input alphabet,
3. a rule for moving between states,
4. a start state,
5. a set of accepting states.

So we write

$$M = (Q, \Sigma, \delta, q_0, F).$$

Definition: Deterministic Finite Automaton

A **deterministic finite automaton**, or DFA, is a 5-tuple

$$M = (Q, \Sigma, \delta, q_0, F),$$

where

- Q is a finite set of states,
- Σ is a finite input alphabet,
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the start state,
- $F \subseteq Q$ is the set of accepting states.

The Running Example: Even Number of 1's

Language we want to recognize:

$$\text{EVEN}_1 = \{w \in \{0,1\}^* : w \text{ contains an even number of 1's}\}.$$

The Running Example: Even Number of 1's

Language we want to recognize:

$$\text{EVEN}_1 = \{w \in \{0,1\}^* : w \text{ contains an even number of 1's}\}.$$

What information must the machine remember?

The Running Example: Even Number of 1's

Language we want to recognize:

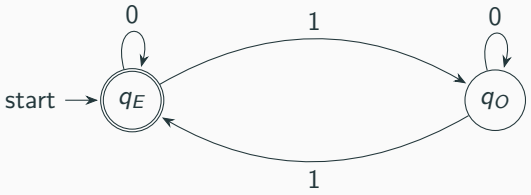
$$\text{EVEN}_1 = \{w \in \{0,1\}^* : w \text{ contains an even number of 1's}\}.$$

What information must the machine remember?

Only whether the number of 1's seen so far is even or odd.

$$q_E : \text{even number of 1's seen}, \qquad q_O : \text{odd number of 1's seen}.$$

The Same Machine as a Diagram



States are not merely circles; they encode the information remembered by the machine.

The Same Machine as Mathematics

$$Q = \{q_E, q_O\}, \qquad \Sigma = \{0, 1\}, \qquad q_0 = q_E, \qquad F = \{q_E\}.$$

The Same Machine as Mathematics

$$Q = \{q_E, q_O\}, \qquad \Sigma = \{0, 1\}, \qquad q_0 = q_E, \qquad F = \{q_E\}.$$

The transition function δ is defined by

δ	0	1
q_E	q_E	q_O
q_O	q_O	q_E

The Same Machine as Mathematics

$$Q = \{q_E, q_O\}, \qquad \Sigma = \{0, 1\}, \qquad q_0 = q_E, \qquad F = \{q_E\}.$$

The transition function δ is defined by

δ	0	1
q_E	q_E	q_O
q_O	q_O	q_E

This table is not a third representation; it is simply a compact definition of δ.

Why Is It Called Deterministic?

For every pair

$$(q, a) \in Q \times \Sigma,$$

there is exactly one next state

$$\delta(q, a) \in Q.$$

Why Is It Called Deterministic?

For every pair

$$(q, a) \in Q \times \Sigma,$$

there is exactly one next state

$$\delta(q, a) \in Q.$$

No guessing.

No ambiguity.

No missing transition.

Exactly one next state.

A DFA is finite memory plus a deterministic update rule.

3B. Acceptance, Rejection, and Languages

Formal Acceptance

Let

$$M = (Q, \Sigma, \delta, q_0, F)$$

be a DFA, and let

$$w = w_1 w_2 \cdots w_n$$

be an input string over Σ.

Formal Acceptance

Let

$$M = (Q, \Sigma, \delta, q_0, F)$$

be a DFA, and let

$$w = w_1 w_2 \cdots w_n$$

be an input string over Σ.

The machine accepts w if there is a sequence of states

$$r_0, r_1, \ldots, r_n$$

such that

1. $r_0 = q_0$,
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ for $0 \leq i \leq n-1$,
3. $r_n \in F$.

What About the Empty String?

The empty string is

$$\varepsilon.$$

It has length zero.

What About the Empty String?

The empty string is

$$\varepsilon.$$

It has length zero.

If the input is ε, the DFA reads no symbols.

What About the Empty String?

The empty string is

$$\varepsilon.$$

It has length zero.

If the input is ε, the DFA reads no symbols.

So it remains in the start state q_0.

What About the Empty String?

The empty string is

$$\varepsilon.$$

It has length zero.

If the input is ε, the DFA reads no symbols.

So it remains in the start state q_0.

Therefore:

A DFA accepts ε exactly when $q_0 \in F$.

Three Similar-Looking Objects

These are different:

Notation	Meaning	Size
ε	the empty string	length 0
$\emptyset$	the empty language	0 strings
$\{\varepsilon\}$	language containing only the empty string	1 string

Three Similar-Looking Objects

These are different:

Notation	Meaning	Size
ε	the empty string	length 0
$\emptyset$	the empty language	0 strings
$\{\varepsilon\}$	language containing only the empty string	1 string

Common mistake:

$$\emptyset \neq \{\varepsilon\}.$$

Dead States

A dead state represents a situation from which success is impossible.

Dead States

A dead state represents a situation from which success is impossible.

Once the machine enters a dead state, it stays there forever.

Dead States

A dead state represents a situation from which success is impossible.

Once the machine enters a dead state, it stays there forever.

Dead states ensure the automaton mathematically remains deterministic and complete.

3C. Worked Examples: Designing States

How to Design a DFA

A good design strategy:

1. Decide what information must be remembered.
2. Make one state for each possible memory situation.
3. Define how each input symbol updates that information.
4. Mark the accepting states.

How to Design a DFA

A good design strategy:

1. Decide what information must be remembered.
2. Make one state for each possible memory situation.
3. Define how each input symbol updates that information.
4. Mark the accepting states.

The hardest step is almost always the first one.

What finite information is enough?

Example 1: Strings Containing 001

Language:

$$L = \{w \in \{0,1\}^* : w \text{ contains } 001 \text{ as a substring}\}.$$

Example 1: Strings Containing 001

Language:

$$L = \{w \in \{0,1\}^* : w \text{ contains } 001 \text{ as a substring}\}.$$

What should the states remember?

Example 1: Strings Containing 001

Language:

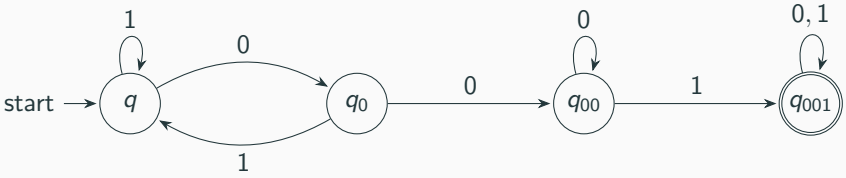
$$L = \{w \in \{0,1\}^* : w \text{ contains } 001 \text{ as a substring}\}.$$

What should the states remember?

The longest suffix of the input so far that is also a prefix of 001.

- nothing useful yet,
- just saw 0,
- just saw 00,
- have seen 001.

DFA for Strings Containing 001



Once 001 appears, the machine stays accepting forever. Notice the 0 loop on q_{00}!

Example 2: Binary Numbers Divisible by 3

Language:

$$L = \{w \in \{0,1\}^* : w \text{ is the binary representation of a number divisible by } 3\}.$$

Example 2: Binary Numbers Divisible by 3

Language:

$$L = \{w \in \{0,1\}^* : w \text{ is the binary representation of a number divisible by } 3\}.$$

At first this looks arithmetic, not automata. But we only need to remember the remainder modulo 3.

$$q_0 : \text{rem } 0, \qquad q_1 : \text{rem } 1, \qquad q_2 : \text{rem } 2.$$

Updating Remainders

If the current value has remainder r modulo 3, and we append bit b, the new value is

$$2r + b \pmod 3.$$

Why? Appending a bit to the right doubles the old binary number and adds the new bit.

Updating Remainders

If the current value has remainder r modulo 3, and we append bit b, the new value is

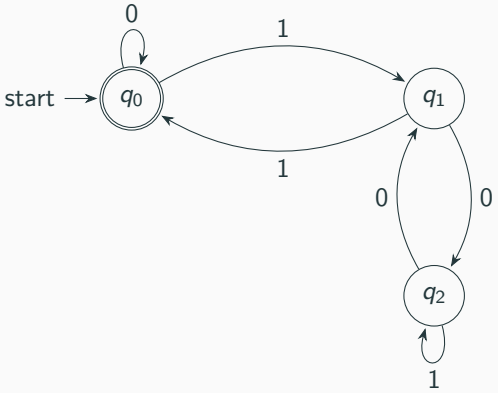
$$2r + b \pmod 3.$$

Why? Appending a bit to the right doubles the old binary number and adds the new bit.

So the transition rule is

$$r \mapsto 2r + b \pmod 3.$$

DFA for Divisibility by 3



Accept when the remainder is 0.

3D. Advanced Examples: Pushing the Limits

Advanced Example 1: The Product Construction Concept

Language over $\Sigma = \{0, 1\}$:

$$L = \{w : w \text{ has an even number of 0's } \textbf{AND} \text{ its length is a multiple of 3}\}.$$

Advanced Example 1: The Product Construction Concept

Language over $\Sigma = \{0, 1\}$:

$$L = \{w : w \text{ has an even number of 0's } \textbf{AND} \text{ its length is a multiple of 3}\}.$$

We must track **two** independent pieces of information simultaneously:

1. Parity of 0's (Even or Odd) $\rightarrow$ 2 possibilities
2. Length modulo 3 (0, 1, or 2) $\rightarrow$ 3 possibilities

Advanced Example 1: The Product Construction Concept

Language over $\Sigma = \{0, 1\}$:

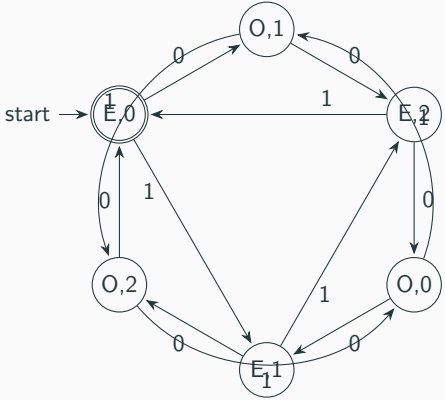
$$L = \{w : w \text{ has an even number of 0's } \textbf{AND} \text{ its length is a multiple of 3}\}.$$

We must track **two** independent pieces of information simultaneously:

1. Parity of 0's (Even or Odd) $\rightarrow$ 2 possibilities
2. Length modulo 3 (0, 1, or 2) $\rightarrow$ 3 possibilities

Total states required: $2 \times 3 = 6$. A state is a pair (p, l).

DFA for Multiple Conditions



This planar embedding shows the intuition behind the **intersection** of two regular languages!

Advanced Example 2: The k-th Symbol from the End

Language:

$$L = \{w \in \{0,1\}^* : \text{the 2nd symbol from the right is a } 1\}.$$

Examples: $10, 011, 110, 0011$. Rejects: $\varepsilon, 0, 1, 100$.

Advanced Example 2: The k-th Symbol from the End

Language:

$$L = \{w \in \{0,1\}^* : \text{the 2nd symbol from the right is a } 1\}.$$

Examples: $10, 011, 110, 0011$. Rejects: $\varepsilon, 0, 1, 100$.

What must the state remember?

Advanced Example 2: The k-th Symbol from the End

Language:

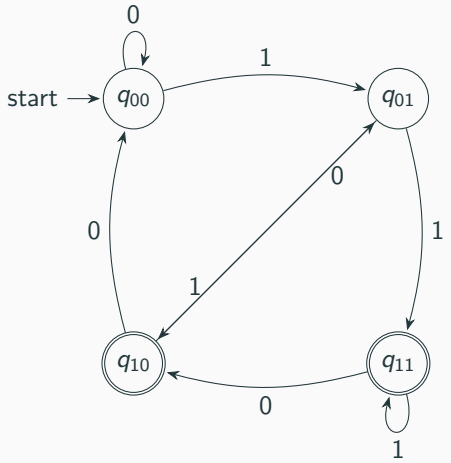
$$L = \{w \in \{0,1\}^* : \text{the 2nd symbol from the right is a } 1\}.$$

Examples: $10, 011, 110, 0011$. Rejects: $\varepsilon, 0, 1, 100$.

What must the state remember? It must act like a sliding window, remembering the **last 2 symbols** seen. States: $q_{00}, q_{01}, q_{10}, q_{11}$.

If we are in q_{01} (last two symbols were $0, 1$) and we read a 0, our new "last two symbols" are $1, 0$. Thus, $q_{01} \xrightarrow{0} q_{10}$.

DFA for the 2nd Symbol from the Right



Note: We treat the initial state q_{00} as pretending we've seen leading zeros. This graph structure is known as a De Bruijn Graph.

Advanced Example 3: Pattern Interference

Language over $\Sigma = \{a, b\}$:

$$L = \{w : w \text{ contains } ab \text{ as a substring, but does } \textbf{not} \text{ contain } ba\}.$$

Advanced Example 3: Pattern Interference

Language over $\Sigma = \{a, b\}$:

$$L = \{w : w \text{ contains } ab \text{ as a substring, but does } \textbf{not} \text{ contain } ba\}.$$

Think about the structure of valid strings:

- They must eventually contain ab.
- Once a b occurs, no a can *ever* follow it (otherwise we get ba).
- Therefore, valid strings look exactly like a^+b^+.

Advanced Example 3: Pattern Interference

Language over $\Sigma = \{a, b\}$:

$$L = \{w : w \text{ contains } ab \text{ as a substring, but does } \textbf{not} \text{ contain } ba\}.$$

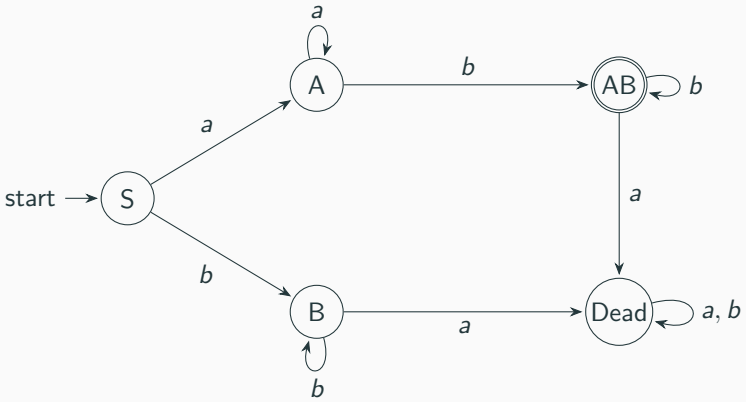
Think about the structure of valid strings:

- They must eventually contain ab.
- Once a b occurs, no a can *ever* follow it (otherwise we get ba).
- Therefore, valid strings look exactly like a^+b^+.

States needed:

1. Starting (seen nothing)
2. Reading a's (safe, waiting for b)
3. Read b prematurely (safe so far, but cannot accept yet, waiting for ba trap)
4. Seen ab (Accepting state, reading more b's is fine)
5. Dead state (Seen ba)

DFA for Pattern Interference



Notice how starting with b means the string can never contain ab without first producing ba.

Advanced Example 4: Lexical Analysis (C-Style Comments)

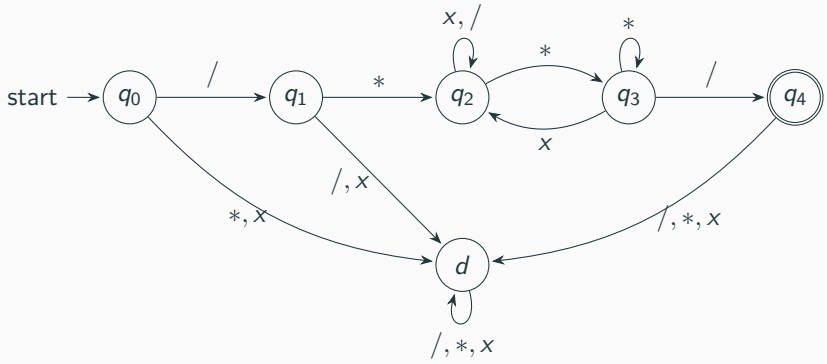
Language over $\Sigma = \{/, *, x\}$ (where x is any other char):

$$L = \{w : w \text{ is exactly one valid block comment starting with } /* \text{ and ending with } */\}.$$

Advanced Example 4: Lexical Analysis (C-Style Comments)

Language over $\Sigma = \{/, *, x\}$ (where x is any other char):

$$L = \{w : w \text{ is exactly one valid block comment starting with } /* \text{ and ending with } */\}.$$



This finite memory tracking is exactly how compilers strip comments from source code!

What You Should Be Able to Do Now

Given a DFA, you should be able to:

- identify $Q, \Sigma, \delta, q_0, F$,
- explain what each state remembers,
- trace an input string and decide if it is accepted,
- describe the language recognized by the DFA.

What You Should Be Able to Do Now

Given a DFA, you should be able to:

- identify $Q, \Sigma, \delta, q_0, F$,
- explain what each state remembers,
- trace an input string and decide if it is accepted,
- describe the language recognized by the DFA.

Given a language description, you should begin by asking:

What finite information is enough?

A DFA is not merely a diagram.

It is a mathematical object that computes using finite memory.